MERN STACK

TRAINING TR-103 REPORT DAY 24

22 July 2026

Overview:

The twenty-fourth day of the MERN Stack training focused on implementing the PUT method to update existing records in MongoDB. I learned how to receive updated data from the client, identify a specific document using route parameters (`req.params`), and modify records using the `updateOne()` method in Mongoose. The session also reinforced the use of `async` and `await` for handling asynchronous database operations and completed the Update operation of CRUD.

Learning Objectives:

- Understand the PUT method.
- Learn how to update data in MongoDB.
- Use `req.params` to identify a document.
- Access updated values using `req.body`.
- Perform update operations using `updateOne()`.
- Handle asynchronous database operations using `async` and `await`.

MongoDB

PUT Method

The PUT method is an HTTP request method used to update existing data on the server. It replaces or modifies the values of an existing record in the database.

Features:

- Updates existing records.
- Sends updated data from client to server.
- Commonly used for editing user information.
- Part of CRUD operations.

Syntax:

```
app.put(path, callback_function);
```

Request Object (req)

The request object (req) contains data sent by the client. It is used to access both route parameters and the updated values.

Common Properties:

- req.params – Retrieves values from the URL.
- req.body – Retrieves updated data sent by the client.

Features:

- Reads client request data.
- Accesses URL parameters.
- Retrieves updated form or JSON data.

Response Object (res)

The response object (res) is used to send the result of the update operation back to the client.

Features:

- Sends success or error messages.
- Returns JSON responses.
- Confirms whether the update was successful.

Syntax:

```
res.send({
statuscode: 1,
msg: "Updated Successfully"
});
```

Async and Await

The async and await keywords are used to perform asynchronous database operations. They ensure that the update operation is completed before sending the response.

Features:

- Handles asynchronous tasks.
- Waits for database operations.
- Improves code readability.
- Works with Promises.

updateOne() Method

The updateOne() method is a Mongoose function used to update a single document that matches a specified condition.

Features:

- Updates one matching document.
- Uses a condition or document ID.
- Modifies only the required fields.
- Returns the update result.

Syntax:

```
Model.updateOne(condition, updatedData);
```

Route Parameters (req.params)

Route parameters are dynamic values passed through the URL. They help identify the specific document that needs to be updated.

Features:

- Retrieves values from the URL.
- Commonly used for document IDs.
- Identifies the record to update.

Example:

```
req.params.id
```

Request Body (req.body)

The req.body object contains the new values sent by the client for updating the document.

Features:

- Retrieves updated user input.
- Accesses JSON or form data.
- Used with the PUT method.

Example:

```
req.body
```

Status Code Response

The server returns JSON responses to indicate the result of the update operation.

Common Responses:

Success

```
res.send({
statuscode: 1,
msg: "Updated Successfully"
});
```

Error

```
res.send({
statuscode: 0,
msg: "Error"
});
```

Task –1

```
const update = (data) => {
    setname(data.Name);
    setage(data.Age);
    setmail(data.Email);
    setid(data._id);
    setflag(true);
};

const up = async () => {
    const result = await fetch(`http://localhost:9000/api/update/${id}`, {
        method: "PUT",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ name, age, mail })
    });

    const res = await result.json();

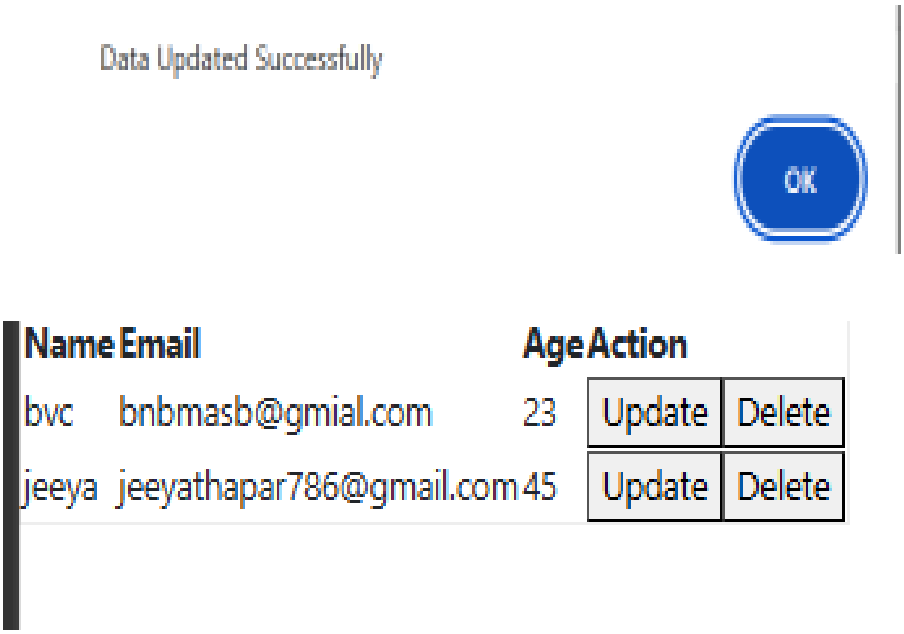
    if (res.statuscode === 1) {
        alert("Data Updated Successfully");

        setname("");
        setage("");
        setmail("");
        setid("");
        setflag(false);

        getdata();
    } else {
        alert("Data not updated");
    }
};
```

```
app.put("/api/update/:id",async(req,res)=>{
    const data=await usermodel.findByIdAndUpdate({_id:req.params.id},{
        $set:{
          Name:req.body.name,
        Email:req.body.mail,
        Age:req.body.age
        }
    });
    if(data){
        res.send({statuscode:1});
    }else{
        res.send({statuscode:0});
    }
})
```

Output:



Conclusion:

By the end of Day 24, I gained a clear understanding of implementing the PUT method in a MERN Stack application. I learned how to update existing records in MongoDB using `updateOne()`, access route parameters through `req.params`, retrieve user input using `req.body`, and perform asynchronous database operations with `async` and `await`. These concepts strengthened my understanding of the Update operation in CRUD and backend API development.